Scripting Languages - Lecture 5: Files, paths and modules (21 Oct)

Lecture overview

Lecture 5 focuses on the essential techniques for managing files, paths, and modules within Python scripting, moving beyond toy data to handle real-world file batches. The session establishes that effective code organization requires separating source code, input data, and results into distinct directories, navigated through file paths. A critical distinction is made between absolute paths, which are machine-specific, and relative paths, which start from the current working directory and ensure portability across different systems. Students learn to manipulate these paths using the `os.path` module to extract components like filenames and extensions, verify file existence, and construct system-compatible strings. File I/O operations utilize the `with open()` statement to safely read or write text files, with specific attention paid to handling line endings and specifying text encodings-primarily UTF-8-to prevent errors when processing non-Western characters or legacy datasets.

The lecture further introduces the command line as a separate interface for file management and demonstrates how to execute shell commands within Jupyter notebooks using exclamation marks or magic commands. To extend Python's built-in capabilities, the session covers importing modules from the standard library, specifically highlighting `glob` for finding files via pattern matching (using wildcards like `*` and `?`) and `pickle` for persisting complex Python objects to disk in binary format. Practical applications include processing batches of files by iterating over sorted lists generated by `glob`, and customizing list sorting using the `key` argument in the `sorted()` function. Finally, the lecture outlines the broader Python data science ecosystem, including libraries like Pandas and NLTK, explains package installation via `pip`, and details course logistics regarding mandatory projects for 6 ECTS students and the weighted grade calculation combining exam and project scores.

Slide 1: faculty of arts

This lecture introduces the fifth session on scripting languages within the Faculty of Arts, focusing specifically on files, paths, and modules. The session is led by Tim Van de Cruys and took place on Wednesday, October 21, 2026.

Slide 2: Working with files

Moving beyond toy data, real applications require processing and saving files stored on disk, often involving large batches like 1,000 files. To manage this effectively, code should be organized structurally by separating source code, input data, and results into distinct directories. This organization relies on understanding paths, which are the specific mechanisms used within programming languages to locate files and navigate directory structures on a computer.

Slide 3: Files and directories

Data is stored as byte sequences in files, which are organized within directories (folders) that can contain both files and nested subdirectories, forming a hierarchical tree structure. The location of any file is defined by its path through this tree, allowing users to avoid disorganized "big piles" by logically separating content such as code, data, and results. As demonstrated with the `ls` command, navigating this structure requires descending through specific directory levels, such as moving from a top-level project folder into nested data or book directories to locate specific files.

Slide 4: Paths

A file path describes a file's location, ranging from a simple filename for files in the current directory to nested directory structures separated by forward slashes (e.g., `data/novels/19thcentury/austen/emma.txt`). To navigate upward to a parent directory, use two dots (`..`). Although Windows typically uses backslashes, Python requires forward slashes to ensure compatibility across all operating systems.

Slide 5: Absolute and relative paths

An absolute path specifies a file's location starting from the root of the file system (e.g., `/home/...` or `C:/...`), which varies by operating system but only functions on the specific computer where it was defined. In contrast, a relative path begins from the current working directory where the program runs, using notations like `data/file.txt` or `../file.txt`. Because absolute paths are tied to individual machine structures, relative paths are preferred in code as they allow the entire directory structure to be copied and executed on other computers without modification.

Slide 6: Where am I?

A relative path is only meaningful when you know your current working directory, which for notebooks is typically the folder containing the notebook file itself. You can verify this location in Python by importing the `os` module and running `os.getcwd()`. If a `FileNotFoundError` occurs despite a correct path, it usually indicates that the code is executing from a different directory than expected, causing the relative reference to fail.

Slide 7: Try it: paths

When running `analysis.ipynb` from the `notebooks/` directory, accessing files outside this folder requires navigating relative paths. To read `emma.txt`, located in a sibling `data/` subdirectory, one must first move up one level using two dots (`..`) to reach the project root before descending into `data/`. Similarly, writing results to the `results/` directory follows the same logic: navigate up from `notebooks/` to the parent directory, then down into `results/` to specify the target file location.

Slide 8: Anatomy of a path

A file path consists of distinct components: the directory (e.g., `data/novels/austen/`), the filename (`emma.txt`), the base name (`emma`), and the extension (`.txt`). Python's `os.path` module provides essential functions to extract or manipulate these parts, such as `basename` to isolate the filename, splitting the extension, `join` to construct paths from directory names, and `exists` to verify file presence. These capabilities are crucial for tasks like sorting files by name or generating output filenames from inputs.

Slide 9: Manipulating paths with os.path

The `os.path` module provides essential functions for manipulating file paths across different systems. Key utilities include `basename` to extract the filename by removing directory components, `splitext` to separate a name from its extension, and `join` to construct paths using the correct system-specific separator. Additionally, `exists` allows scripts to verify a file's presence before attempting to open it, preventing errors during file operations.

Slide 10: Reading a file

To read a file, use the `with open('document.txt') as in_file` statement, which automatically closes the file upon completion. You can iterate line-by-line using a nested `for` loop, where each line is a string containing the newline character `\n`; apply `line.strip()` to remove this trailing whitespace and other leading/trailing spaces. Alternatively, `in_file.read()` loads the entire file content as a single string rather than processing it line by line.

Slide 11: Reading and writing

To read and write files, use two `with` statements to open an input file (default 'r' mode) and an output file ('w' mode overwrites existing content). Iterate through the input line by line, process each string (e.g., replacing characters), and explicitly add newlines when writing since `write()` does not append them automatically. This approach works only for plain text files; binary formats like Word or PDF cannot be processed this way because they encode layout data rather than raw text.

Slide 12: Text encoding

Text files rely on specific encodings to translate sequences of bits into readable characters, a process critical for displaying non-Western languages and emojis correctly. Computers use these encoding schemes to ensure the proper character appears on screen; failing to apply the correct encoding when reading or saving text results in garbled output, similar to viewing a web page with mismatched settings.

Slide 13: Text encoding

Always use the Unicode character set with UTF-8 encoding, which supports diverse languages and emojis and serves as the default in Python. Using the correct encoding when reading or writing files prevents programs from misinterpreting bytes and displaying incorrect characters. While UTF-8 is standard, issues may arise with older datasets or Windows programs, requiring you to explicitly specify the encoding via a keyword argument to resolve compatibility problems.

Slide 14: Text encoding: specifying the encoding

Attempting to load a text file in an incorrect encoding, such as opening a Latin-1 file without specification, causes Python to raise a `UnicodeDecodeError` because it defaults to UTF-8. This issue frequently arises with older datasets or on Windows systems where legacy encodings persist. The solution is to explicitly specify the correct encoding parameter within the `open()` function, ensuring the interpreter correctly decodes the specific character sequences present in the file.

Slide 15: The command line

The command line (also called terminal, shell, or console) is a textual interface for issuing operating system commands to manage files and run programs like Python. While Linux and Mac use the Terminal with Unix shells, Windows utilizes Anaconda Prompt or PowerShell, where commands may differ slightly despite cross-platform compatibility efforts. Crucially, the command line is a separate language from Python that operates at a higher level to control the OS; entering a Python interpreter via a command exits the shell environment entirely.

Slide 16: Basic commands

The command line executes shell-specific commands like `pwd`, `ls` (or `dir` on Windows), and `cd` to navigate directories, distinguishing itself from Python code execution which requires the explicit `python` command. Essential operations include creating directories with `mkdir`, managing files via `cp`, `mv`, or `rm`, and inspecting file contents using `head`. Efficiency is enhanced by using Tab for automatic filename completion and the up arrow to repeat previous commands, while paths function relative to the current working directory just as they do in Python.

Slide 17: The command line from a notebook

In a Jupyter notebook, prepend an exclamation mark (!) to execute standard command-line operations like listing files or downloading data, distinguishing them from Python code. Because each ! command runs in a fresh shell, directory changes using !cd do not persist; instead, use the %cd magic command to permanently alter the notebook's working directory. This distinction ensures that file system navigation remains effective across subsequent cells while allowing flexible integration of external tools.

Slide 18: Try it: the command line

Starting from the `project/` directory, this exercise challenges you to identify the specific command-line instructions needed to list files within `novels/`, navigate into `notebooks/`, and subsequently display the first lines of `emma.txt`. The professor notes that these operations can be performed either directly in a terminal or within a notebook using an exclamation mark prefix. This practical scenario reinforces understanding of directory traversal and file inspection commands relative to the current working directory.

Slide 19: From files to tools

Modules are reusable collections of code that extend Python's built-in functionality, allowing developers to leverage existing solutions rather than writing everything from scratch. This section introduces specific standard library modules: `os.path` for path manipulation, `glob` for finding files using patterns, and `pickle` for persistently saving objects. The lecture also distinguishes between importing entire modules versus specific functionalities and notes that while some modules are included in the standard distribution, others require separate installation.

Slide 20: Importing modules

To extend functionality beyond built-in capabilities, Python uses the `import` statement to access external modules. You can either import an entire module using `import <module>` or selectively import a specific component with `from <module> import <function>`. These two syntaxes provide flexible options for integrating additional code into your scripts.

Slide 21: Importing modules

To extend beyond built-in functionality, Python allows importing modules from its standard library using two primary syntaxes: `import <module>` or `from <module> import <function>`. Importing the whole module requires prefixing usage with the module name (e.g., `math.pi`), which clarifies the source but is less convenient. Alternatively, importing a specific part (e.g., `sqrt`) allows direct function calls without the module prefix, making it more concise though less explicit about origin.

Slide 22: The Python standard library

The Python standard library follows a "batteries included" philosophy, providing numerous useful modules like `math`, `random`, and `statistics` that are included with the distribution and available for immediate import. While the full documentation is extensive, this session focuses on three specific examples: `os.path` for manipulating file paths, `glob` for finding files by pattern, and `pickle` for saving Python objects to disk.

Slide 23: Finding files by pattern

Before reading files, you often need to locate them using globbing, which matches a set of files against a search pattern. The most common operators are the asterisk (*), representing zero or more characters, and the question mark (?), matching exactly one character. For instance, a pattern like `data/*.txt` automatically selects all files with a `.txt` extension within the `data` subdirectory, regardless of their specific names, by generalizing the filename portion.

Slide 24: Finding files by pattern

Before reading files, globbing allows matching a set of files using search patterns within the `glob` module. The asterisk (`*`) matches zero or more characters, while the question mark (?) matches exactly one character, enabling efficient retrieval of multiple files like `scripting_practical?.ipynb` or all `.txt` files in a directory without typing each name individually. These same patterns function identically on the command line, returning a list of matching filenames for further processing.

Slide 25: Processing multiple files

The `glob` module efficiently processes batches of files by selecting them via a pattern, such as `'data/*.txt'`, and iterating through the results in a loop. Since `glob` returns filenames in an arbitrary order, wrapping it with `sorted()` ensures a predictable sequence when order matters. This common Python pattern involves opening each file individually within the loop to perform identical operations, like reading content and counting words, before printing the results for every document.

Slide 26: Sorting with a key function

The slide demonstrates basic list sorting in Python using the `sorted()` function, which arranges elements in ascending alphabetical order by default. Applying this to a list of names like `['Charlie', 'Alice', 'Bob']` produces the ordered result `['Alice', 'Bob', 'Charlie']`. While the slide title references sorting with a key function, the provided example illustrates the standard behavior without an explicit key argument, establishing the baseline for how strings are naturally compared and sorted.

Slide 27: Sorting with a key function

The `sorted()` function arranges list elements alphabetically by default, as shown when `['Charlie', 'Alice', 'Bob']` becomes `['Alice', 'Bob', 'Charlie']`. By passing a `key` argument, such as the built-in `len` function, the sorting criterion changes to element length rather than alphabetical order. This transforms the same list into `['Bob', 'Alice', 'Charlie']`, ordering items from shortest to longest string.

Slide 28: Sorting with a key function

The `key` argument in sorting applies a function to each item before comparison, allowing custom criteria like string length instead of alphabetical order. For example, `sorted(names, key=len)` orders names by character count (3, 5, then 7), while any custom function or the `reverse=True` flag can further modify the sort behavior.

Slide 29: Saving Python objects with pickle

The `pickle` module enables saving complex Python objects, such as dictionaries or lists, directly to disk in binary mode (`'wb'`, `'rb'`) rather than as human-readable text. By using `pickle.dump()` to write and `pickle.load()` to read, you can persistently store intermediate results like a translation dictionary, ensuring the exact object structure is preserved across Python sessions instead of being lost when the program closes.

Slide 30: The Python data science ecosystem

The Python data science ecosystem comprises specialized libraries for distinct analytical tasks. Jupyter provides interactive notebooks, while Spacy and NLTK handle natural language processing, with NLTK specifically noted for education. Data visualization is covered by Matplotlib for general plots and Seaborn for statistical graphics. Core analysis relies on Pandas for tabular data, Scikit-learn for machine learning classification and clustering, and gensim for topic modeling using LDA.

Slide 31: Installing packages

Most required packages are pre-installed within the Anaconda distribution, with Spacy and gensim being notable exceptions. While conda packages can be installed via Anaconda Navigator, more advanced or numerous packages require pip, a command-line tool that can be executed directly in notebooks using the `%pip install` magic command (e.g., `%pip install spacy`).

Slide 32: Practical session / reading

Students should access Practical Session 4 via the code-along notebook available on Toledo to re-run lecture examples at home. Additionally, reading Chapters 9 and 10 of Python for Everybody is required, as this completes the coverage of Chapters 1-10, which collectively establish the fundamentals of Python.

Slide 33: Project

The project is mandatory exclusively for students enrolled in the 6 ECTS course version, such as those in Digital Humanities or Sociology, while it is not required for the 4 ECTS track. Participants must choose one of three options: analyzing provided word rating datasets, exploring their own data, or applying discussed programming techniques to a personal research question like corpus construction or traffic stream analysis. Due by early January, this component is integral to the 6 ECTS grade; failure to submit results in an NA score.

Slide 34: Grade calculation

Grade calculation differs by course size: a 4 ECTS course relies solely on a final exam scored out of 20, whereas a 6 ECTS course combines the 4 ECTS exam component (weighted at two-thirds) with a project (weighted at one-third). For instance, in a 6 ECTS scenario, an exam score of 12/20 and a project score of 15/20 are calculated as $(12 \times \frac{2}{3}) + (15 \times \frac{1}{3})$, resulting in a final grade of 13/20.

Key points to remember

The lecture further introduces the command line as a separate interface for file management and demonstrates how to execute shell commands within Jupyter notebooks using exclamation marks or magic commands. To extend Python's built-in capabilities, the session covers importing modules from the standard library, specifically highlighting `glob` for finding files via pattern matching (using wildcards like `*` and `?`) and `pickle` for persisting complex Python objects to disk in binary format. Practical applications include processing batches of files by iterating over sorted lists generated by `glob`, and customizing list sorting using the `key` argument in the `sorted()` function. Finally, the lecture outlines the broader Python data science ecosystem, including libraries like Pandas and NLTK, explains package installation via `pip`, and details course logistics regarding mandatory projects for 6 ECTS students and the weighted grade calculation combining exam and project scores.